



EDUCACIÓN
SECRETARÍA DE EDUCACIÓN PÚBLICA



TECNOLÓGICO
NACIONAL DE MÉXICO



TECNOLÓGICO NACIONAL DE MÉXICO

INSTITUTO TECNOLÓGICO DE TIJUANA

SUBDIRECCIÓN ACADÉMICA

DEPARTAMENTO DE SISTEMAS Y COMPUTACIÓN

PERIODO: ENERO - JUNIO 2026

Carrera:

Ingeniería en sistemas

Materia:

Patrones de Diseño

Tema:

Patron Estado

Alumnos:

Felix Sandoval Josue Guadalupe 21211940

Docente:

Maribel Guerrero Luis

Fecha:

06/05/2026

¿Qué es el patrón State?

El **patrón State** o **patrón Estado** es un patrón de diseño de software que permite que un objeto **cambie su comportamiento dependiendo del estado en el que se encuentra**.

En lugar de tener muchos if, else o switch para preguntar:

```
if (estado == "BuscandoConductor") {  
    // hacer algo  
} else if (estado == "ConductorAsignado") {  
    // hacer otra cosa  
}
```

El patrón State separa cada estado en una **clase diferente**.

Diagrama UML

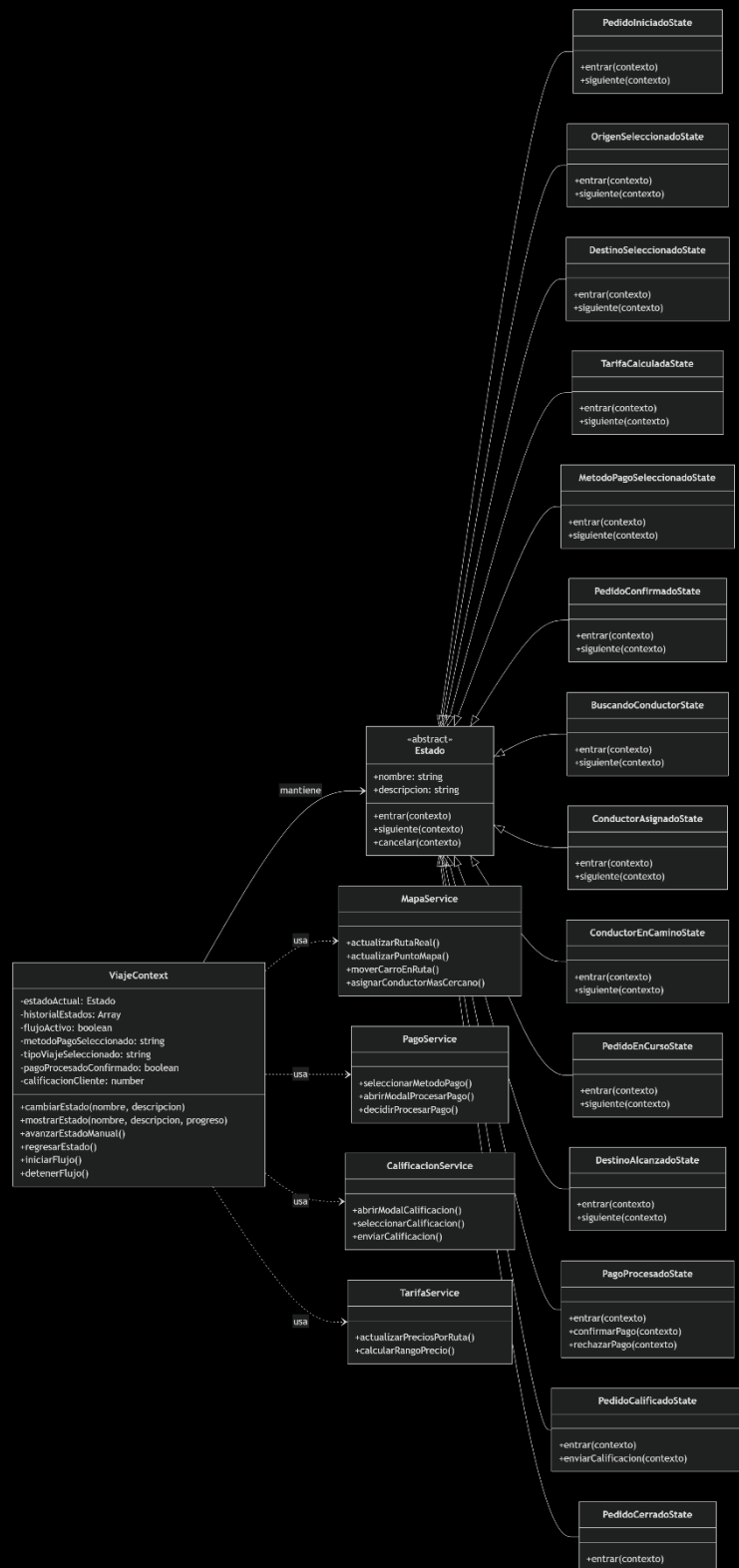
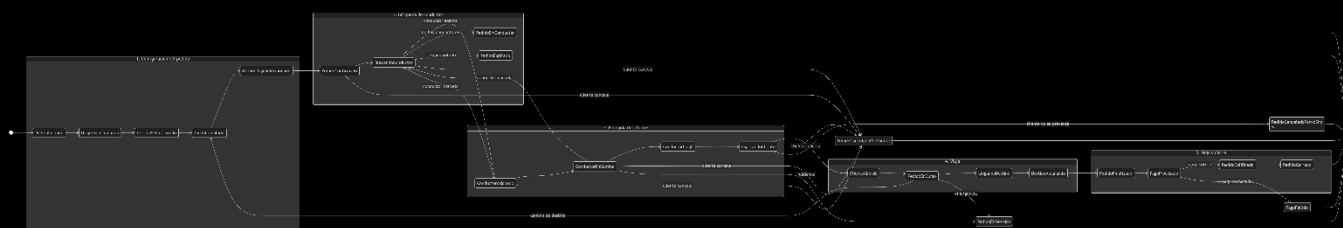


Diagrama de estados



Código

```
// state-pattern.js
// Implementación independiente del Patrón Estado (versión por clases).
// Este archivo contiene un `Context` (RideContext) y varios `State`
// concretos que encapsulan el comportamiento dependiente del estado.
// Está pensado para importarse o incluirse en la página cuando se
// quiera una versión más estructurada del patrón.
"use strict";

// Interfaz base para los estados
/**
 * Nota: estas clases son ligeras y están pensadas para usarse localmente
 * o a través del objeto exportado `window.RideStatePattern` que se
 * expone al final del archivo. Evitamos mutaciones posteriores.
 */
class State {
  constructor(context) {
    this.context = context;
  }

  // Método llamado al entrar en el estado
  enter() {}

  // Método llamado para ejecutar la acción principal del estado
  handle() {}

  // Método opcional para salir del estado
  exit() {}
}

// Contexto que mantiene el estado actual y delega comportamiento
class RideContext {
  constructor() {
    this.state = null;
    this.history = [];
  }

  setState(stateInstance) {
    if (this.state && typeof this.state.exit === 'function') {
      this.state.exit();
    }

    if (this.state) this.history.push(this.state.constructor.name);
    this.state = stateInstance;
  }
}
```

```

    if (this.state && typeof this.state.enter === 'function') {
        this.state.enter();
    }
}

// Delegar la acción principal al estado actual
handle() {
    if (!this.state) return;
    return this.state.handle();
}
}

// Estados concretos
class PedidoIniciadoState extends State {
    enter() {
        console.log('Estado: Pedido iniciado');
    }

    handle() {
        // Lógica específica: validar orígenes/destinos, preparar UI
        console.log('Preparando solicitud de viaje...');
        // transición de ejemplo a OrigenSeleccionado
        this.context.setState(new OrigenSeleccionadoState(this.context));
    }
}

class OrigenSeleccionadoState extends State {
    enter() {
        console.log('Estado: Origen seleccionado');
    }

    handle() {
        // Lógica: esperar selección de destino
        console.log('Esperando destino...');
        this.context.setState(new DestinoSeleccionadoState(this.context));
    }
}

class DestinoSeleccionadoState extends State {
    enter() {
        console.log('Estado: Destino seleccionado');
    }

    handle() {
        // Lógica: calcular tarifa y pasar a confirmación

```

```

        console.log('Calculando tarifa...');
        this.context.setState(new PedidoConfirmadoState(this.context));
    }
}

class PedidoConfirmadoState extends State {
    enter() {
        console.log('Estado: Pedido confirmado');
    }

    handle() {
        // Lógica: buscar conductor y asignarlo
        console.log('Buscando conductor...');
        this.context.setState(new BuscandoConductorState(this.context));
    }
}

class BuscandoConductorState extends State {
    enter() {
        console.log('Estado: Buscando conductor');
    }

    handle() {
        // Simular asignación y transición
        console.log('Asignando conductor...');
        this.context.setState(new ConductorAsignadoState(this.context));
    }
}

class ConductorAsignadoState extends State {
    enter() {
        console.log('Estado: Conductor asignado');
    }

    handle() {
        console.log('Conductor en camino');
        this.context.setState(new ConductorEnCaminoState(this.context));
    }
}

class ConductorEnCaminoState extends State {
    enter() {
        console.log('Estado: Conductor en camino al origen');
    }
}

```

```
handle() {
  // Ejemplo: al llegar, cliente aborda
  console.log('Conductor llegó al origen');
  this.context.setState(new ClienteAbordoState(this.context));
}
}

class ClienteAbordoState extends State {
  enter() {
    console.log('Estado: Cliente abordó');
  }

  handle() {
    console.log('Pedido en curso');
    this.context.setState(new PedidoEnCursoState(this.context));
  }
}

class PedidoEnCursoState extends State {
  enter() {
    console.log('Estado: Pedido en curso');
  }

  handle() {
    console.log('Llegando a destino');
    this.context.setState(new DestinoAlcanzadoState(this.context));
  }
}

class DestinoAlcanzadoState extends State {
  enter() {
    console.log('Estado: Destino alcanzado');
  }

  handle() {
    console.log('Finalizando pedido');
    this.context.setState(new PedidoFinalizadoState(this.context));
  }
}

class PedidoFinalizadoState extends State {
  enter() {
    console.log('Estado: Pedido finalizado');
  }
}
```



```
    handle() {
      console.log('Procesando pago y cerrando');
      // Fin del flujo: no más transiciones automáticas
    }
  }

// Export helpers for manual testing from console or integration
window.RideStatePattern = Object.freeze({
  RideContext,
  State,
  PedidoIniciadoState,
  OrigenSeleccionadoState,
  DestinoSeleccionadoState,
  PedidoConfirmadoState,
  BuscandoConductorState,
  ConductorAsignadoState,
  ConductorEnCaminoState,
  ClienteAbordoState,
  PedidoEnCursoState,
  DestinoAlcanzadoState,
  PedidoFinalizadoState
});

// Ejemplo rápido (comentar si se integra en la app principal):
// const ctx = new RideContext();
// ctx.setState(new PedidoIniciadoState(ctx));
// ctx.handle(); // Ejecuta flujo de ejemplo
```

Programa ejecutado

Solicita un viaje

Punto de partida

Origen: 32.52076, -117.09349

X

Destino

Destino: 32.52496, -117.07323

X

Tipo de viaje

5.1 km - 5 min estimados

UberX

Viaje económico para hasta 4 personas.

MXN 100-130

Comfort

Auto más amplio y conductores mejor calificados.

MXN 150-190

UberXL

Más espacio para hasta 6 personas.

MXN 200-255

Metodo de pago

Tarjeta **** 4242

V

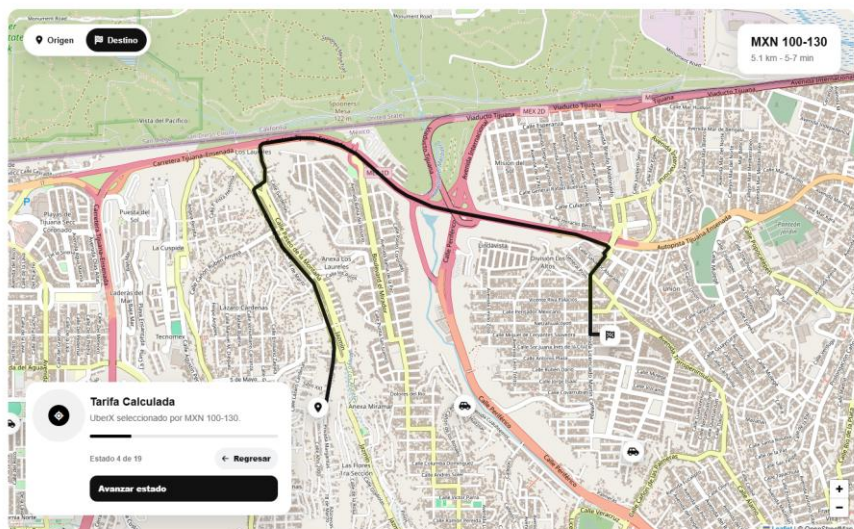
Buscar

Cancelar viaje

Cambiar destino

Emergencia

Reembolso



Solicita un viaje

Punto de partida

Origen: 32.52076, -117.09349

X

Destino

Destino: 32.52496, -117.07323

X

Tipo de viaje

5.1 km - 5 min estimados

UberX

Viaje económico para hasta 4 personas.

MXN 100-130

Comfort

Auto más amplio y conductores mejor calificados.

MXN 150-190

UberXL

Más espacio para hasta 6 personas.

MXN 200-255

Metodo de pago

Efectivo

V

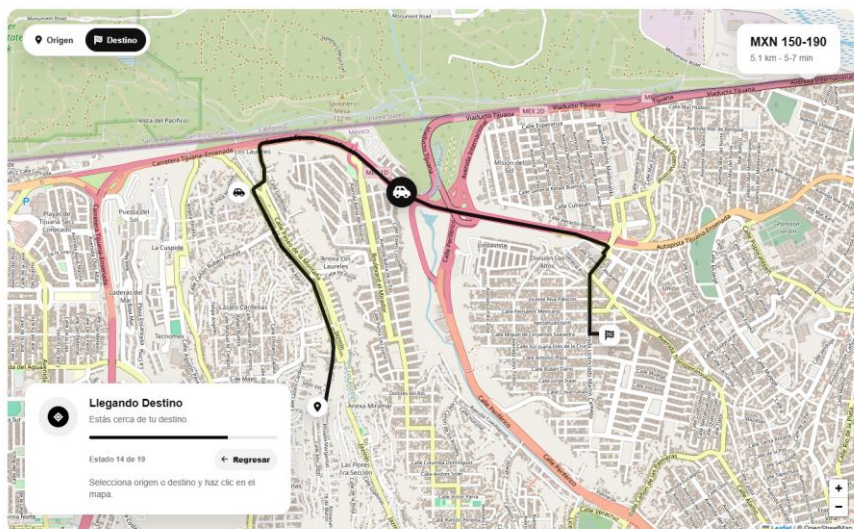
Buscar

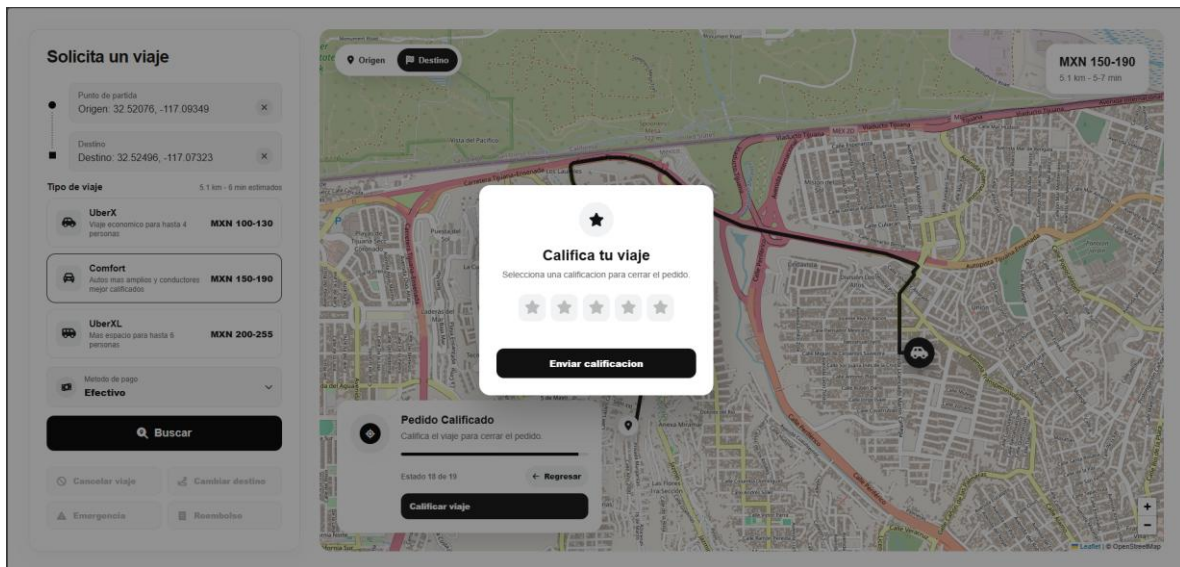
Cancelar viaje

Cambiar destino

Emergencia

Reembolso





Conclusión:

El patrón **State** es útil porque permite organizar mejor el comportamiento de un objeto según el estado en el que se encuentre. En lugar de usar muchos condicionales, cada estado se maneja en una clase diferente, lo que hace que el código sea más limpio, ordenado y fácil de mantener. Este patrón es ideal para sistemas donde un proceso pasa por varias etapas, como un pedido, una compra o un viaje en una aplicación.